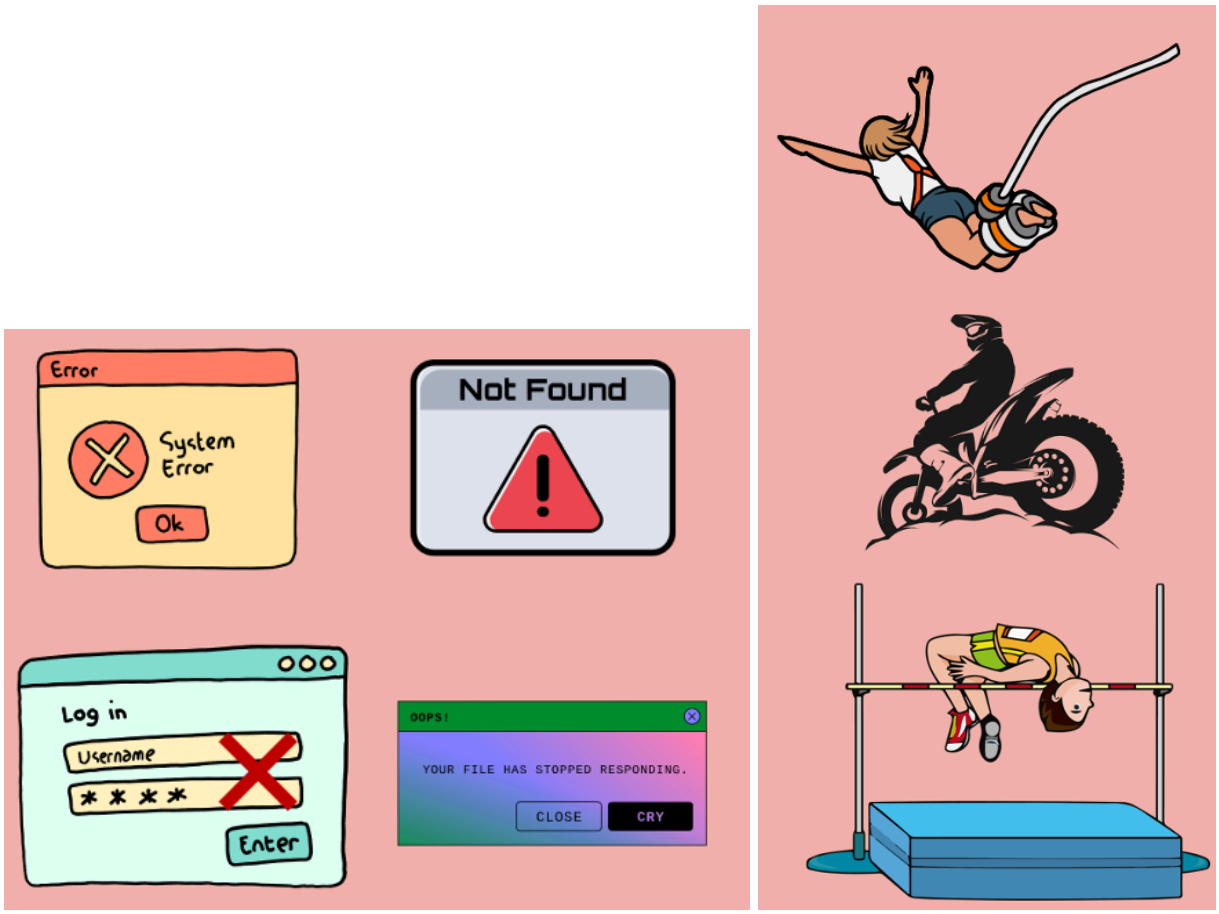
Exception Handling In Java:



- Irrespective of your programming proficiency, it's impossible to exercise complete control over everything. There is always a possibility of things going extremely wrong, like System/Network errors, File Not Found, Invalid input, file not responding, etc.
- When you write a risky method, you should be prepared to handle the potential problems that might arise. This is very similar to the safety measures that we take during adventure activities.
- **Exception Handling** in Java is one of the powerful *mechanisms for handling runtime errors,* so that the normal flow of the application can be maintained.

Real Life Analogy

- Imagine you are **driving a car**
- While driving, many unexpected things can happen:

Situation	Programming Meaning
Petrol finished	Exception
Tyre Puncture	Exception
Traffic police stop	Exception
Engine failure	Exception

- Just like in real life, **software must handle unexpected situations**.
- That is why Java provides an **Exception Handling Mechanism**.

What is an Exception?

Definition

- In Java, an exception is an unwanted or unexpected event that occurs during the execution of a program, i.e., at run time, that disrupts the normal flow of the program's execution.
- Technically, in Java, exceptions are objects of some Java classes that will be created by the JVM at runtime whenever any Logical error occurs in our Java application.
- The core advantage of exception handling is **to maintain the normal flow of the application**. An exception normally disrupts the normal flow of the application. That is why we need to handle exceptions.

Example:

Demo.java:

```
package com.hdfc;
import java.util.Scanner;
public class Demo {
```

```
        public static void main(String[] args) {

                System.out.println("Program starts...");

                Scanner sc = new Scanner(System.in);

                System.out.println("Enter number 1: ");
                int num1 = sc.nextInt();

                System.out.println("Enter number 2: ");
                int num2 = sc.nextInt();

                int result = num1 / num2;
                System.out.println(result);

                sc.close();

                System.out.println("Program ends...");
        }
}
```

What happens here?

1. The program compiles successfully.
2. If we pass num2 as 0, our program will terminate abnormally without executing the last statement.
3. During execution, the JVM detects **division by zero**
4. JVM creates an object of **ArithmeticException**

Types of Errors in Java:

1. Syntax Errors

- These occur due to **incorrect syntax or improper environment setup**.

  Example:

    - Missing semicolon
    - Incorrect method declaration

- They are detected by the **compiler during compilation**.

2. Logical Errors (Runtime Errors)

- These occur during program execution.

  Example:

    - Division by zero
    - Null reference access

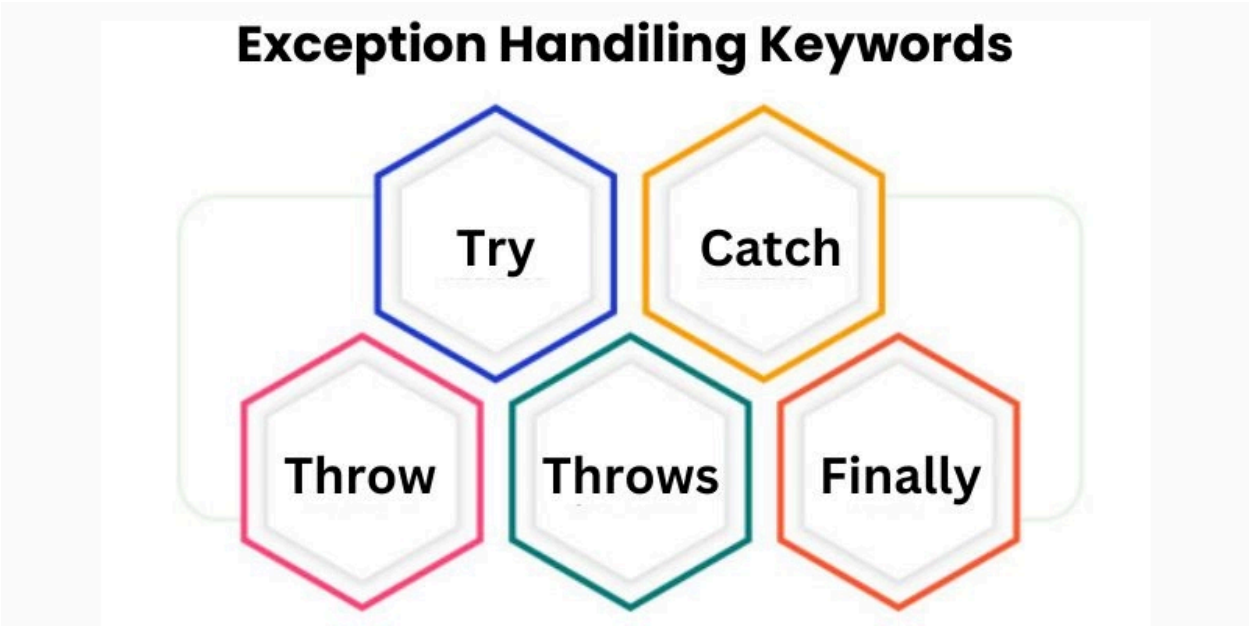
- Whenever a logical error occurs, the program will be terminated, and control comes out of the program unconditionally without mentioning from which part of the program it has come out.

What Happens When an Exception Occurs?

1. JVM detects a runtime problem.
2. JVM creates an **object of the corresponding Exception class**.
3. If the exception is **not handled**, it is returned to the JVM.
4. JVM receives that exception class object and **terminates the program abnormally**.

Exception Handling

- **Exception Handling** is the process of **handling exceptions to prevent abnormal program termination**.
- It allows the program to **continue execution normally**.
- Java provides **five keywords** for exception handling:



Keyword	Description
try	The "try" keyword is used to specify a block where we should place an exception code. It means we can't use a try block alone. The try block must be followed by either catch or finally.
catch	The "catch" block is used to handle the exception. It must be preceded by a try block, which means we can't use a catch block alone. It can be followed by finally block later.
finally	The "finally" block is used to execute the necessary code of the program. It is executed whether an exception is handled or not.
throw	The "throw" keyword is used to throw an exception.
throws	The "throws" keyword is used to declare exceptions. It specifies that an exception may occur in the method. It doesn't throw an exception. It is always used with a method signature.

try-catch Block

- The **try block** contains code that may generate exceptions.

- The **catch block** handles the exception.

Syntax:

```
try
{
  risky code
}
catch(ExceptionType  e)
{
  handling code
}
```

Example:

```
package com.hdfc;
import java.util.Scanner;
public class Demo {
	public static void main(String[] args) {

		System.out.println("Program starts...");
		Scanner sc = new Scanner(System.in);

		try {
			System.out.println("Enter number 1: ");
			int num1 = sc.nextInt();
			System.out.println("Enter number 2: ");
			int num2 = sc.nextInt();
			int result = num1 / num2;
			System.out.println(result);
			System.out.println("Inside try");


		} catch (ArithmeticException ae) {
			System.out.println("Division by Zero is not allowed..");
			ae.printStackTrace();
		}
		sc.close();
		System.out.println("Program ends...");
```

```
        }
    }
```

Here,

- If the exception occurs in the try block.
- The exception object is passed to the catch block.
- The catch block handles the exception.

Uses of Exception Handling

1. Ensures **normal termination of the program**.
2. Helps identify **where the logical error occurred**.
3. Prevents **JVM from terminating the entire program**.
4. Only the faulty **try block terminates**, not the whole program.

Important Rules of try-catch

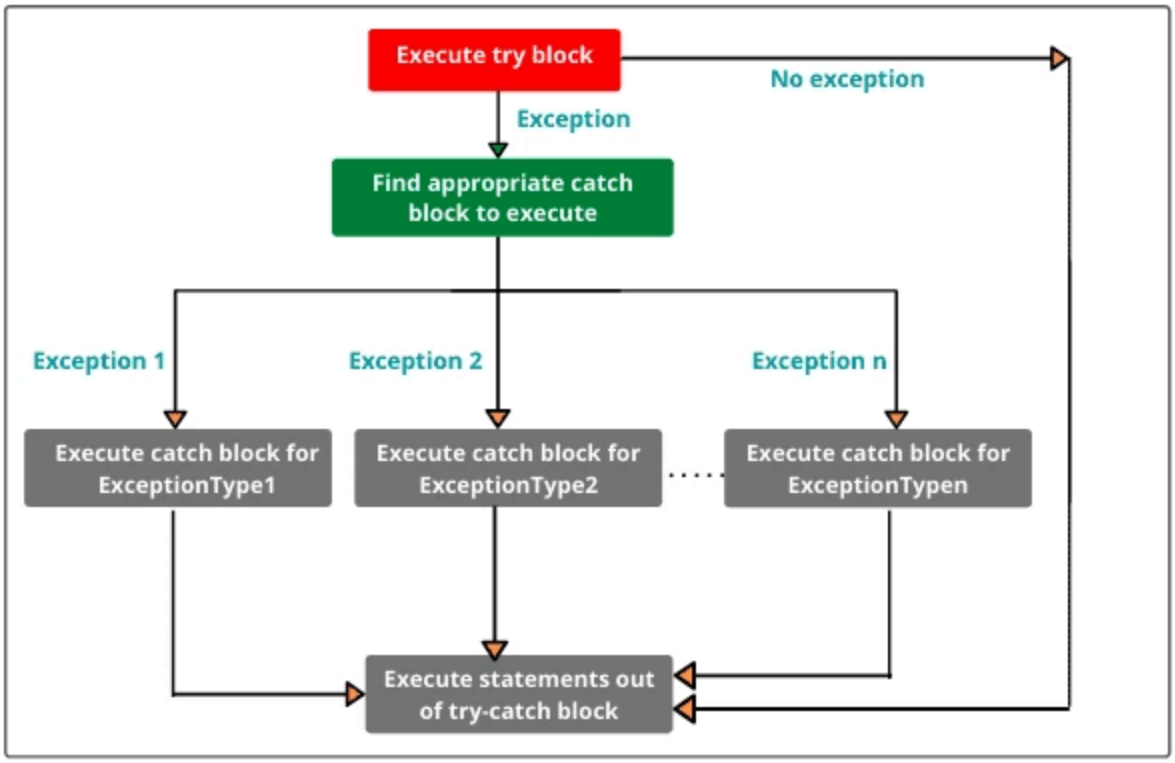
1. Once control enters the **catch block**, it does not return to the try block.
2. If **no exception occurs**, the catch block will not execute.
3. Every **try block must have at least one catch or finally block**.
4. A try block can have **multiple catch blocks**.

Multiple Catch Blocks Example:

Demo.java:

```
package com.hdfc;
```

```java
import java.util.InputMismatchException;
import java.util.Scanner;
public class Demo {


	public static void main(String[] args) {

		System.out.println("Program starts..");
		Scanner sc = new Scanner(System.in);

		try {
			String s1 = null;
			System.out.println("Enter num1: ");
			int n1 = sc.nextInt();
			System.out.println("Enter num1: ");
			int n2 = sc.nextInt();
			int x = n1 / n2;
			if (x > 10) {
				s1 = "Welcome";
			}

			System.out.println(s1.toUpperCase());
		}

		catch (InputMismatchException ab) {
			ab.printStackTrace();
		}
		catch (ArithmeticException ae) {
			ae.printStackTrace();
		}
		catch (NullPointerException np) {
			np.printStackTrace();
		}

		System.out.println("End of main()");
	}
}
```



Problem with Multiple Catch Blocks

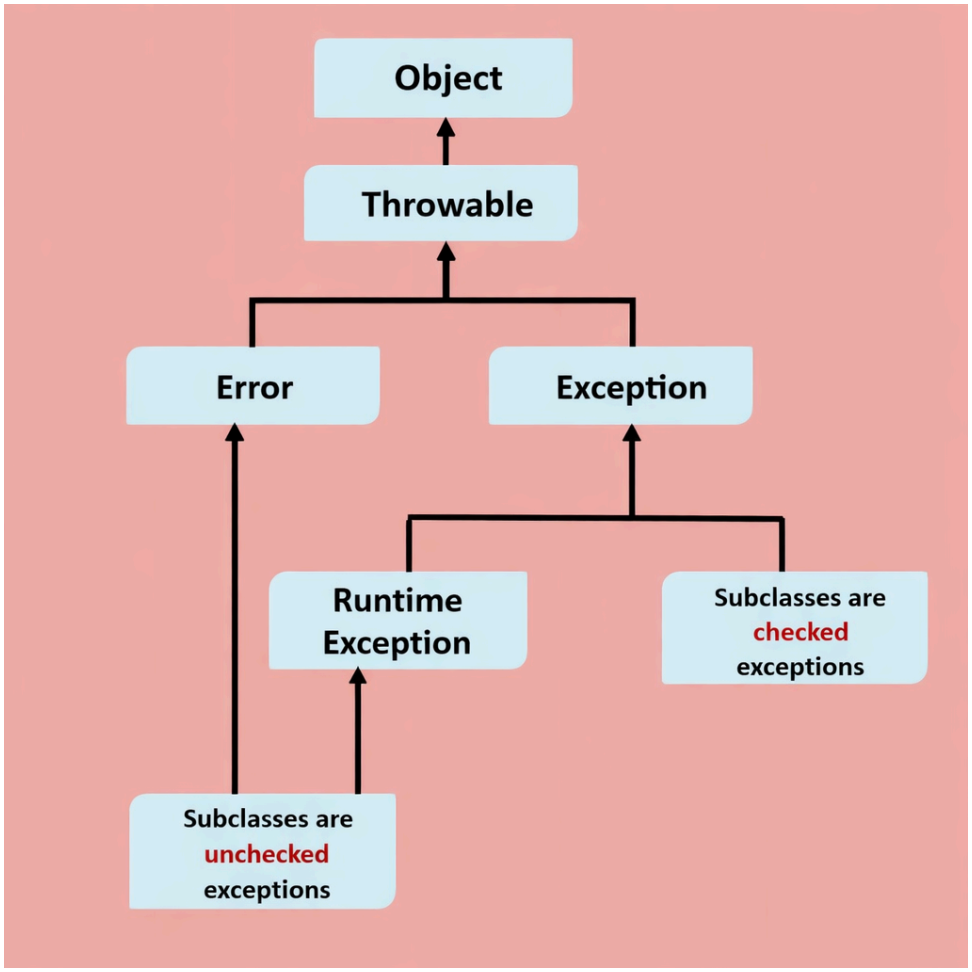
- If multiple statements in the try block may generate different exceptions, we may need **many catch blocks**, which becomes difficult to manage.

Solution:

- Use the **Exception superclass**.

```
catch(Exception e)
```

Exception class Hierarchy:



Exception: Logical errors that can be handled. They are recoverable problems.

Examples:

IOException

SQLException

NullPointerException

Error: Serious problems that should not be handled by programmers.

Examples of Errors:

- OutOfMemoryError

- StackOverflowError

Why We Should Not Handle Error Class

- These are **serious system-level problems** handled by the JVM internally.
- Handling them may hide serious problems and is **dangerous**.

Throwable Class

- `Throwable` is the **superclass of all errors and exceptions**.
- Both try and catch can only work with:
  - `Throwable`
  - Subclasses of `Throwable`

Example:

```
catch(Throwable e)
```

- But it is **not recommended** because it also catches Errors.

Preferred:

```
catch(Exception e)
```

Order of Catch Blocks

- When multiple catch blocks exist, they are checked **from top to bottom**.

Example (**Incorrect**):

```
try {
}
```

```
catch(Exception e) {
}
catch(ArithmeticException e) {
}
```

- This causes a **compile-time error** because the second catch block becomes **unreachable**.

Correct order:

```
try {
}
catch(ArithmeticException e) {
}
catch(Exception e) {
}
```

Some of the important methods of the Throwable class:

1. **public void printStackTrace();**
    - Prints detailed exception information
2. **public String getMessage();**
    - Returns the exception message

Checked vs Unchecked Exceptions

Checked Exceptions

- A **Checked Exception** is an exception that is **checked by the compiler at compile time**.
- If your code might cause a checked exception, **the compiler forces you to handle it**.

  You must either:

  1. Handle it using **try-catch**
  2. Declare it using **throws.**
- Otherwise, the program **will not compile**.

Think of **Checked Exception like Airport Security**

- Before boarding a flight, you **must pass a security check**.

  If you don't pass the check:

  - You cannot board the flight.

Similarly,

If you do not handle a **Checked Exception**

- The compiler will not allow the program to run.

Example of Checked Exceptions

Common Checked Exceptions:

- `IOException`
- `SQLException`
- `FileNotFoundException`
- `ClassNotFoundException`
- `InterruptedException`

Example 1: FileNotFoundException

```
package com.hdfc;
import java.io.FileReader;
public class Demo
{
   public static void main(String[] args)
   {
      FileReader fr = new FileReader("abc.txt");
   }
}
```

Compiler Error

- Because **FileNotFoundException is a checked exception**.
- The compiler forces you to handle it.

Solution 1: Using try-catch

```
package com.hdfc;
import java.io.FileReader;
public class Demo {
	public static void main(String[] args) {
		try {
			FileReader fr = new FileReader("abc.txt");
			System.out.println("File opened successfully");
		} catch (Exception e) {
			System.out.println("File not found");
		}
	}
}
```

Solution 2: Using throws

```
package com.hdfc;
import java.io.FileNotFoundException;
import java.io.FileReader;
public class Demo
{
  public static void main(String[] args) throws FileNotFoundException
  {
     FileReader fr = new FileReader("abc.txt");
  }
}
```

Here we are telling the compiler:

- "If an exception occurs, I will not handle it here."

Unchecked Exceptions

- An **Unchecked Exception** is an exception that is **NOT checked by the compiler at compile time**.
- The compiler **does not force you to handle it**.
- These exceptions occur **during program execution (runtime)**.

Think Unchecked Exception is like **slipping on a banana peel**

- Nobody checks before walking.

  But suddenly:

  Boom! You slipped!

Similarly,

- Unchecked exceptions **suddenly occur at runtime**.

The compiler **does not warn you**.

Common Unchecked Exceptions

- These belong to the **RuntimeException class**

Examples:

- `ArithmeticException`
- `NullPointerException`
- `ArrayIndexOutOfBoundsException`
- `NumberFormatException`
- `IllegalArgumentException`

Example: ArithmeticException

```
class Test
{
  public static void main(String[] args)
  {
     int a = 10;
     int b = 0;
     int c = a / b;
     System.out.println(c);
  }
}
```

Output

```
Exception in thread "main" java.lang.ArithmeticException: / by zero
```

- The compiler allowed it, but the **runtime failed**.

Rule

- All exceptions that extend:

  RuntimeException

- are **Unchecked Exceptions**
- All other exceptions are **Checked Exceptions**

When Should We Use Checked Exceptions?

- Checked Exceptions are used when the **program can recover**.

  Examples:

  - File reading
  - Database connection
  - Network communication

- Because the problem **can be handled**.

When Do Unchecked Exceptions Occur?

- Unchecked exceptions usually happen due to **programming mistakes**.

  Examples:

  - Dividing by zero
  - Accessing an invalid index
  - Using a null reference

- These are **developer errors**.
- Handling the unchecked exceptions is optional, but good programmers **still handle them**.

Example:

```
class Test
```

```
{
  public static void main(String[] args)
  {
     try
     {
        int arr[] = {10,20,30};
        System.out.println(arr[5]);
     }
     catch(Exception e)
     {
        System.out.println("Something went wrong!");
     }
  }
}
```

The throws Keyword

- The `throws` is used in the method declaration.
- It informs the caller that the method may generate an exception.

Example

```
void fun1(int i, int j) throws ArithmeticException, NullPointerException {

}
```

Meaning:

- The method may generate these exceptions.
- The caller can either:
    1. Handle them using try-catch
    2. Delegate them further using throws

Example of Checked Exception

```
class B {
  void funA() throws ClassNotFoundException {
     Class.forName("A");
  }
```

```
}
```

- Here, Class.forName() may generate **ClassNotFoundException**, which is a **checked exception**.

Therefore, the method must either:

- Handle it with try-catch
  or
- Declare it using `throws`.

throws with the constructor

- Constructors can also declare exceptions.

Example:

```
class Test {

  int z = 20;

  Test() throws Exception {
     int i = 10;
     int j = 0;
     int x = i / j;
  }

  public static void main(String[] args) {
     try {
        Test t = new Test();
     }
     catch(Exception e) {
        e.printStackTrace();
     }
  }
}
```

Important Point

- If a constructor throws a checked exception, we cannot create an object as an instance variable directly because instance variable initialization cannot contain such executable exception-handling logic.

Method Overriding Rules with Exceptions

Case 1

If the **superclass method does not declare any exception**

Subclass method:

- Cannot declare checked exceptions
- Can declare unchecked exceptions

Case 2

If the **superclass method declares an exception**

Subclass method can declare:

1. Same exception
2. Child class exception
3. No exception

But cannot declare **parent exception**.

Superclass Constructor Exception

When a subclass object is created:

1. Superclass constructor executes first

2. Then the subclass constructor executes

If superclass constructor throws an exception:

Example:

```
class A {
  A() throws Exception {
     System.out.println("Inside A()");
  }
}

class B extends A {
  public static void main(String[] args) {
     try {
        B b1 = new B();
     }
     catch(Exception e) {
        System.out.println(e);
     }
  }
}
```

finally Block

- The **finally block always executes**, whether an exception occurs or not.

Example

```
try {
  System.out.println("Try block");
}
catch(Exception e) {
  System.out.println(e);
}
finally {
  System.out.println("Finally block");
}
```

Why finally is Needed

Example:

```
try {
  Connection con = getConnection();
  // operations
  con.close();
}
```

- If an exception occurs before `con.close()`, the connection remains open.
- This causes **resource leakage**.

Solution:

```
finally {
  con.close();
}
```

try with finally (without catch)

- It is possible, but **not recommended**.

```
try {
}
finally {
}
```

- finally **will NOT execute if:**

System.exit()
JVM crash
Power failure

User-Defined Exception

- A **Custom Exception** (also called **User-Defined Exception**) is an exception that **we create ourselves in Java** to represent **specific business errors** in our application.
- Java already provides many exceptions, such as:
    - `ArithmeticException`

- `NullPointerException`
- `IOException`
- `ArrayIndexOutOfBoundsException`

- But sometimes **these built-in exceptions are not meaningful enough** for our program logic.

Example Situation

Suppose we are developing a **banking system**.

Rules:

- Minimum balance must be **₹1000**

If a user tries to withdraw money, leaving a balance below ₹1000, we should show:

InsufficientBalanceException: Minimum balance must be ₹1000

This is where **Custom Exception** is useful.

Why Do We Need Custom Exceptions?

- Custom exceptions help us to:
  1. Provide **meaningful error messages**
  2. Represent **business logic errors clearly**
  3. Improve **code readability**
  4. Make debugging easier
  5. Handle **application-specific conditions**

How to Create a Custom Exception in Java

- Creating a custom exception is **very simple**.
- We just need to **extend an existing Exception class**.

  Two options:

  Exception

  RuntimeException

  So the **syntax** is:

```
class MyException extends Exception
{
}
```

Types of Custom Exceptions

1. **Checked Custom Exception**: If the parent class is Exception
2. **Unchecked Custom Exception**: If the parent class is RuntimeException

Example 1: Creating a Checked Custom Exception

- Checked exceptions must be **handled using try-catch or throws**.
- Suppose we want to allow voting only for age **>= 18**.

InvalidAgeException.java

```
class InvalidAgeException extends Exception
{
  InvalidAgeException(String message)
  {
     super(message);
  }
}
```

- Here, super(message) passes the message to the **Exception class constructor**.

Use the Custom Exception:

VotingSystem.java:

```
class VotingSystem
{
  static void checkAge(int age) throws InvalidAgeException
  {
     if(age < 18)
     {
        throw new InvalidAgeException("You are not eligible to vote");
     }
     else
     {
        System.out.println("You are eligible to vote");
     }
  }
  public static void main(String[] args)
  {
     try
     {
        checkAge(16);
     }
     catch(InvalidAgeException e)
     {
        System.out.println(e.getMessage());
     }
  }
}
```

Example 2: Creating an Unchecked Custom Exception

- Unchecked exceptions extend **RuntimeException**.
- These are **not checked by the compiler**.

InvalidSalaryException.java

```
class InvalidSalaryException extends RuntimeException
{
  InvalidSalaryException(String message)
  {
     super(message);
  }
}
```

Using InvalidSalaryException

Company.java

```
class Company
{
  static void checkSalary(int salary)
  {
     if(salary < 10000)
     {
        throw new InvalidSalaryException("Salary cannot be less than 10000");
     }
     System.out.println("Valid Salary");
  }
  public static void main(String[] args)
  {
     checkSalary(5000);
  }
}
```

Difference Between throw and throws.

throw	throws
Used the inside method	Used in method declaration

throw	throws
Throws a specific exception object	Declares possible exceptions
Used for custom exceptions	Used for informing the caller

Important Java 7 Features

1. Multiple Exceptions in One Catch

```
catch(ClassNotFoundException | IOException e) {
  e.printStackTrace();
}
```

Rules:

- Use | (pipe symbol)
- The variable becomes **implicitly final.**
- Cannot combine exceptions of the **same hierarchy**

Example (**Invalid**):

```
catch(Exception | IOException e)
```

2. Try-With-Resources

- Introduced in **Java 7**.
- Automatically closes resources.
- Resources must implement: java.lang.AutoCloseable

Example:

```
try(FileInputStream fis = new FileInputStream("file.txt")) {

}
```

- The file will automatically close.

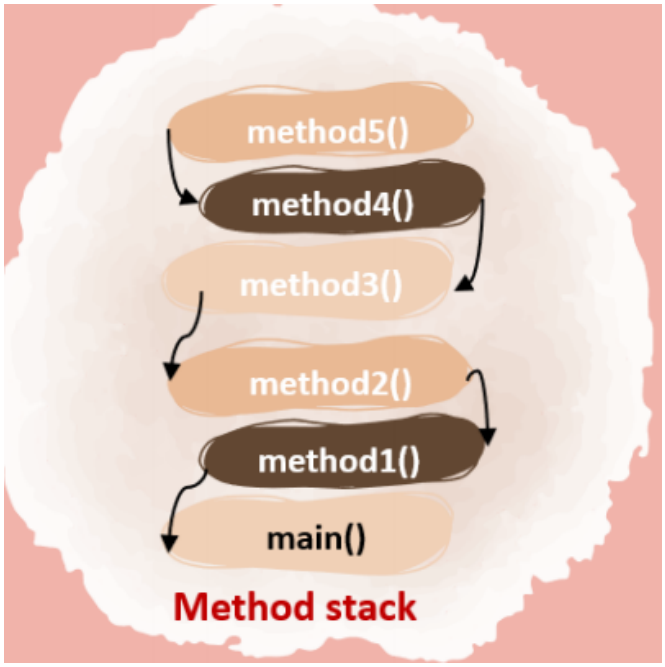
Example:

```
import java.util.Scanner;
class Test {
  public static void main(String[] args) {
     try(Scanner sc = new Scanner(System.in))
     {
        int x = sc.nextInt();
        System.out.println(x);
     }
  }
}
```

Exception Propagation

- When an exception occurs in a method and is not handled, it is passed to the **calling method**.

Example:



- Exceptions travel upward until they are handled.

- If an exception occurs in method5() and if none of the methods in the call stack didn't handle the exception, then the exception will keep propagating till it reaches the end of the method stack, which is the main() method.

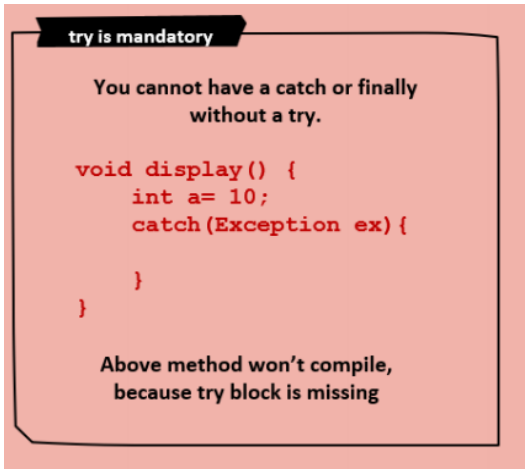
Nested try Block

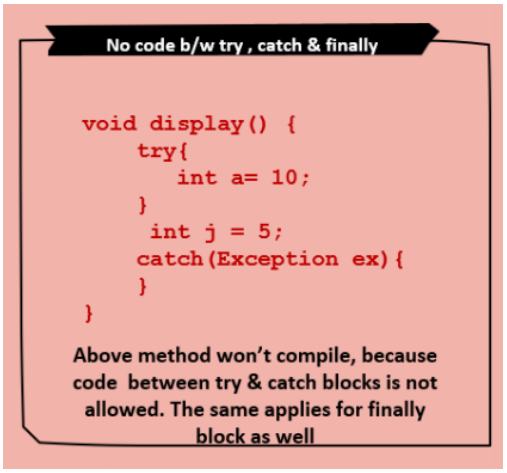
- A try block inside another try block.

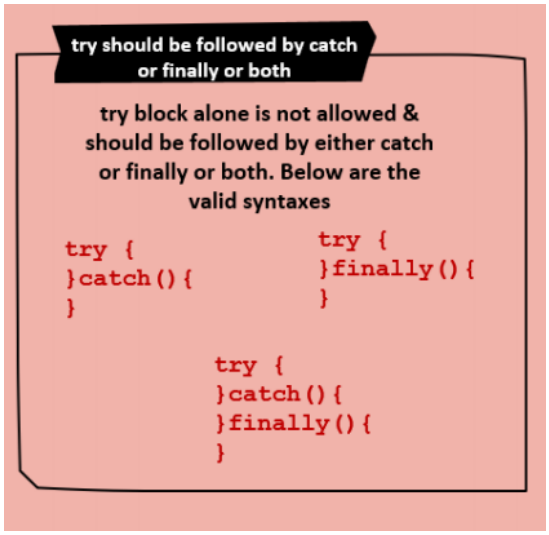
Example

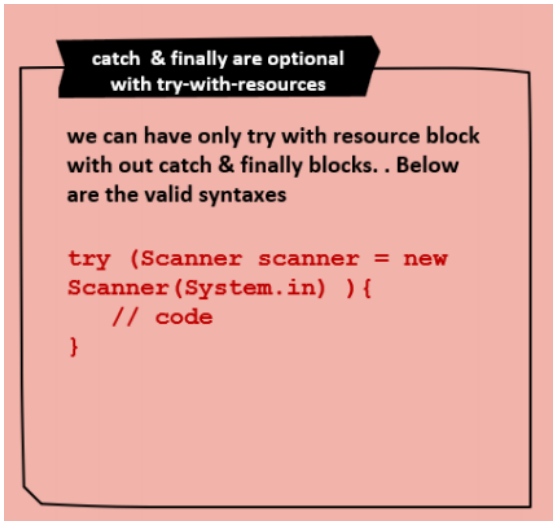
```
try
{
  try
  {
     int x = 10/0;
  }
  catch(ArithmeticException e)
  {
     System.out.println(e);
  }
}
catch(Exception e)
{
  System.out.println("Outer catch");
}
```

Rules while handling exceptions:









Student Task:

1. **Predict the Output:**

```
class Test{
  public static void main(String[] args){
    try{
      System.out.println("A");
      int x = 10/0;
      System.out.println("B");
    }
    catch(Exception e){
      System.out.println("C");
```

```
        }
        System.out.println("D");
    }
}
```

Options

A. A B C D
B. A C D
C. A B D
D. Compile error

2. **Predict the Output:**

```
class Test{
    public static void main(String[] args){
        try{
            int arr[] = new int[5];
            System.out.println(arr[5]);
        }
        catch(ArithmeticException e){
            System.out.println("Arithmetic");
        }
        catch(Exception e){
            System.out.println("Exception");
        }
    }
}
```

Options

A. Arithmetic
B. Exception
C. Runtime error
D. Compile error

3. **Identify the correct statement.**

Options

A. try block must have a catch
B. The try block must have finally
C. try must have either catch or finally
D. try must have both catch and finally

4. **What will happen?**

```
try{
}
System.out.println("Hello");
catch(Exception e){
}
```

Options

A. Correct program
B. Compile error
C. Runtime error
D. Logical error

- **Catch must come immediately after try.**

5. **Which class is the superclass of all Exceptions?**

Options

A. Object
B. Throwable
C. RuntimeException
D. Error

6. **What will be the output?**

```
class Test{
  public static void main(String[] args){
     try{
        return;
     }
     finally{
```

```
            System.out.println("Finally");
        }
    }
}
```

Options

A. Nothing
B. Finally
C. Compile error
D. Runtime error

7. **What happens if we write?**

```
catch(Exception | ArithmeticException e)
```

Options

A. Valid syntax
B. Runtime error
C. Compile error
D. Logical error

8. **What will happen?**

```
try{

}
```

Options

A. Valid program
B. Compile error
C. Runtime error
D. Logical error

9. **What will be the output?**

```
class Test{
  public static void main(String[] args){
    try{
      int x = 10/0;
    }
    catch(ArithmeticException e){
      System.out.println("A");
    }
    finally{
      System.out.println("B");
    }
  }
}
```

Options

A. A
B. B
C. A B
D. Compile error

10. What will happen?

```
class Test{
  public static void main(String[] args){
    try{
      System.out.println(10/0);
    }
    finally{
      System.out.println("Finally");
    }
  }
}
```

Options

A. Finally
B. ArithmeticException + Finally
C. Compile error
D. Runtime error

11. Which of the following statements is true?

Options

A. try block must have a catch
B. The try block must have finally
C. try must have catch or finally
D. try must have both

12. What will be the Output:

```
class Test {
  static int test() {
     try {
        return 1;
     }
     finally {
        return 2;
     }
  }
  public static void main(String[] args) {
     System.out.println(test());
  }
}
```

Options:

A. 1
B. 2
C. Compilation Error
D. Runtime Error.

13. What will be the Output:

```
class Test {
  static int test() {
     int x = 10;
```

```
        try {
            return x;
        }
        finally {
            x = 20;
        }
    }
    public static void main(String[] args) {
        System.out.println(test());
    }
}
```

Options:

A. 10
B. 20
C. Compilation Error
D. Runtime Error.

14. What will be the Output:

```
class Test {
    public static void main(String[] args) {
        try {
            try {
                int x = 10/0;
            }
            catch(NullPointerException e) {
                System.out.println("Inner Catch");
            }
        }
        catch(Exception e) {
            System.out.println("Outer Catch");
        }
    }
}
```

Options:

A. Inner Catch
B. Outer Catch

C. Compilation Error
D. Runtime Error.

15. What will be the Output:

```
class Test {
  public static void main(String[] args) {
    try {
      System.out.println("Try");
      System.exit(0);
    }
    finally {
      System.out.println("Finally");
    }
  }
}
```

Options:

A. Try
B. Finally
C. Compilation Error
D. Runtime Error.

16. What will be the Output:

```
class Test {
  public static void main(String[] args) {
    for(int i=0;i<2;i++){
      try{
        break;
      }
      catch(Exception e) {
       System.out.println("Catch");
      }
      finally{
        System.out.println("Finally");
      }
    }
  }
```

```
}
```

Options:

A. Catch
B. Finally
C. Compilation Error
D. Runtime Error.

Coding Challenges:

1. Question

Problem Statement

- You are developing a **simple banking application**. The bank has a rule that **every account must maintain a minimum balance of ₹1000**.
- A customer is allowed to withdraw money only if the **remaining balance after withdrawal is greater than or equal to ₹1000**.
- If a customer tries to withdraw money that violates this rule, the program should throw a **custom exception called `InsufficientBalanceException`**.

Requirements

Your program should perform the following tasks:

1. Create a **custom exception class** named:

   InsufficientBalanceException

2. Create a **BankAccount class** with the following:

   - A variable `balance`
   - A method `withdraw(int amount)`

3. The `withdraw()` method should:
    - Allow withdrawal if the remaining balance ≥ ₹1000
    - Otherwise throw `InsufficientBalanceException`
4. In the **main method**:
    - Create a bank account with an initial balance.
    - Ask the user to enter the withdrawal amount.
    - Handle the exception using **try-catch**

Expected Behaviour

Case 1: Valid Withdrawal

Current Balance: 5000
Enter amount to withdraw: 2000

Withdrawal successful
Remaining Balance: 3000

Case 2: Invalid Withdrawal

Current Balance: 5000
Enter amount to withdraw: 4500

Exception: Minimum balance of ₹1000 must be maintained

Solution:

InsufficientBalanceException.java

```
class InsufficientBalanceException extends Exception
{
  InsufficientBalanceException(String message)
  {
     super(message);
  }
}
```

BankAccount.java:

```
class BankAccount
{
  int balance;
  BankAccount(int balance)
  {
     this.balance = balance;
  }
  void withdraw(int amount) throws InsufficientBalanceException
  {
     if(balance - amount < 1000)
     {
        throw new InsufficientBalanceException(
           "Minimum balance of ₹1000 must be maintained."
        );
     }
     balance = balance - amount;
     System.out.println("Withdrawal successful");
     System.out.println("Remaining Balance: " + balance);
  }
}
```

BankApplication.java:

```
import java.util.Scanner;
public class BankApplication
{
  public static void main(String[] args)
  {
     Scanner sc = new Scanner(System.in);
     BankAccount account = new BankAccount(5000);
     System.out.println("Current Balance: 5000");
     System.out.print("Enter amount to withdraw: ");
     int amount = sc.nextInt();
     try
     {
        account.withdraw(amount);
     }
     catch(InsufficientBalanceException e)
     {
```

```
            System.out.println("Exception: " + e.getMessage());
        }
        sc.close();
    }
}
```

2. Question:

Problem Statement: Array Element Access

- Write a Java program that stores **5 numbers in an array**.
- Ask the user to enter an index to access an element of the array.
- If the user enters an index outside the valid range, the program should handle the exception and display a friendly message.

3. Question:

Problem Statement: Password Validation System

You are building a **user registration system**.

The password must satisfy the following rules:

- Minimum length **8 characters**
- Must contain at least **one digit**

If the password does not follow these rules, throw a custom exception:

InvalidPasswordException

Example Output

Enter password: abc123

Exception: Password must contain at least 8 characters